

UNIVERSIDAD DE SANTIAGO DE
COMPOSTELA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

Optimización en el acceso a nubes de puntos de 3D mediante la utilización de octrees y coordenadas polares

Autor/a:

Manuel Ramallo Blanco

Tutores:

Francisco M. Fernández Rivera

Miguel Yermo García

Grado en Ingeniería Informática

Junio 2026

Trabajo de Fin de Grado presentado en la Escuela Técnica Superior de
Ingeniería de la Universidad de Santiago de Compostela para la obtención del
Grado en Ingeniería Informática



D. (Nome da titora ou do tutor), Profesor/a do Departamento de Electrónica e Computación da Universidade de Santiago de Compostela, e **D. (Nome da cotirora ou do cotitor)**, Profesor/a do Departamento de Electrónica e Computación da Universidade de Santiago de Compostela,

INFORMAN:

Que a presente memoria, titulada (*Título do traballo*), presentada por **D. (Nome da autora ou do autor do traballo)** para superar os créditos correspondentes ao Traballo de Fin de Grao da titulación de Grao en Enxeñaría Informática, realizouse baixo nosa tutoría no Departamento de Electrónica e Computación da Universidade de Santiago de Compostela.

E para que así conste aos efectos oportunos, expiden o presente informe en Santiago de Compostela, a (Data):

Titor/a,

Cotitor/a,

Alumno/a,

(Nome do titor/a) (Nome do cotitor/a) (Nome do alumn/a)

Agradecimentos

Se se quiere pór algún agradecimento, este vai aquí.

Resumen

En este trabajo se describe y se presenta una solución de mejora de eficiencia temporal en el acceso a nubes de puntos 3D utilizando como estructura de datos *octrees* lineales. El estudio consiste en el reordenamiento espacial de puntos para tratar de explotar la capacidad de poda a la hora de encontrar vecindades. Se evalúan resultados temporales usando diferentes técnicas de reordenamiento, operando tanto con las coordenadas cartesianas como con las coordenadas polares de los puntos. Los resultados experimentales muestran un impacto positivo en la eficiencia temporal bajo determinadas circunstancias, a cambio de un gasto en memoria utilizada.

Indice

1. Introducción	1
2. Estado de conocimiento del problema a abordar	5
3. Materiales	9
4. Metodología	11
5. Pruebas	13
6. Discusión de los resultados	15
7. Exemplos (eliminar capítulo na versión final)	17
7.1. Un exemplo de sección	17
7.1.1. Un exemplo de subsección	17
7.1.2. Otro exemplo de subsección	17
7.2. Exemplos de figuras e cadros	18
7.3. Exemplos de referencias á bibliografía	18
7.4. Exemplos de enumeracións	19
8. Conclusións e posibles ampliacións	21
A. Manuais técnicos	23
B. Manuais de usuario	25
C. Licenza	27
Bibliografía	29

Índice de figuras

7.1. Esta é a figura de tal e cal.	18
--	----

Índice de cuadros

3.1. Mapa de nubes utilizadas en las evaluaciones experimentales. . . .	9
7.1. Esta é a táboa de tal e cal.	18

Capítulo 1

Introducción

Las nubes de puntos 3D son una representación de una realidad, como un objeto o un terreno, mediante un conjunto de puntos en el espacio tridimensional, estando cada punto caracterizado por sus coordenadas X, Y y Z así como otras propiedades como el color o la intensidad. Entre las aplicaciones más importantes de la elaboración de estas nubes están la cartografía, la elaboración de modelos tridimensionales en CAD de piezas fabricadas, la metrología, la arqueología o la conducción autónoma. La tecnología predominante para la captura de estos puntos es la LiDAR (Light Detection And Ranging).

Una de las tareas más comunes al trabajar sobre nubes de puntos 3D es la búsqueda de vecindades alrededor de un punto dado. Una vecindad se define como el conjunto de puntos de puntos que están dentro de una figura geométrica determinada en el espacio con radio R y con el centro en el punto para el cual se están buscando vecinos. Esta combinación de figura geométrica de y radio se conoce como kernel de búsqueda. Otro método alternativo de análisis de vecindades es buscar los K puntos más cercanos al elemento de partida, técnica llamada KNN (*k nearest neighbors*).

Un gran problema a nivel de hardware de este proceso sobre nubes de puntos es que estos datos se recogen sin ningún tipo de estructura ni orden basado en localidad. Este fenómeno se junta con el hecho de que estas nubes pueden tener millones de puntos almacenados. Por eso, se necesitan métodos eficientes para acelerar este lento proceso de acceso a puntos cercanos.

Los *Octrees* y los *KDTrees* son estructuras jerárquicas ampliamente utilizadas para lidiar con este problema de ineficiencia. Consisten en dividir la nube de puntos de manera recursiva en zonas próximas en el espacio físico. Los *Octrees* son estructuras donde cada nodo se divide exactamente 8 hijos, dividiendo cada dimensión a la mitad (solo se usa en espacios tridimensionales). Los puntos pertenecen al nodo hoja que los contiene en su interior. Por otra parte, los *KD-Trees* son árboles binarios que dividen cada nodo interno solamente según una dimensión. En cada nivel se elige un eje y se traza un hiperplano que divide a los puntos en dos nodos. De esta forma, este árbol es generalizable para cualquier

espacio de dimensiones.

El rendimiento de estas estructuras depende directamente de la distribución de los puntos. Por eso es muy útil la aplicación técnicas de reordenamiento espacial basadas en curvas de llenado de espacio (SFC). Las curvas de llenado son una representación mediante una línea continua de un cuadrado unidad bidimensional o, de forma general, un hipercubo N dimensional. En el caso de las nubes de puntos 3D nos interesa mapear los puntos tridimensionales a una representación unidimensional. El objetivo de la aplicación de curvas de llenado es mejorar la coherencia entre la localidad espacial de los puntos y su correspondiente localidad en memoria, con el fin de mejorar el rendimiento en esos accesos a memoria a la hora de calcular vecindades. Para lograr este reordenamiento, en la literatura destacan las curvas de Morton y las curvas de Hilbert, que implementan diferentes estrategias para proyectar los puntos en un índice lineal unidimensional.

En este trabajo se usará como base el modelo de *Octrees* propuesto por [1] donde se aplican las reordenaciones espaciales de Morton y Hilbert sobre diferentes implementaciones de *Octrees*, implementando tanto la estructura clásica basada en punteros como una versión lineal. Sobre esa implementación de algoritmo se tratará de optimizar todavía más el acceso a puntos vecinos. Para ello se empleará la técnica algorítmica de la poda.

Esta estrategia, de forma general, consiste en descartar o 'podar' regiones del espacio (nodos) o conjuntos de datos que matemáticamente se puede demostrar que no satisfacen las condiciones necesarias para ser solución. Su aplicación en este contexto es ser capaz de conocer *a priori* qué partes del *Octree* no pueden contener ningún vecino para una búsqueda dada. Esta técnica es, de hecho, el motivo por el que los *Octrees* (y los KDTrees) son estructuras ideales para almacenar nubes de puntos 3D, es muy sencillo podar nodos comprobando si sus límites están fuera del alcance del kernel de vecindad. Sin embargo, en este trabajo se explorará la poda a nivel de puntos dentro de un nodo, es decir, ser capaz de descartar puntos pertenecientes a un nodo hoja sin tener que evaluarlos individualmente. Para conseguir esta tarea se usarán reordenamientos de puntos a nivel de hoja siguiendo diversos criterios basados en coordenadas cartesianas con ejes X, Y, Z como definidores de los puntos, así como sistemas alternativos de de representaciones tridimensionales: sistemas esféricos y cilíndricos, que se tratan de expansiones de las coordenadas polares.

Las coordenadas polares son un sistema de coordenadas bidimensional donde cada punto es caracterizado por una distancia radial y un ángulo con respecto del origen. Las coordenadas cilíndricas añaden el componente de la altura a las propiedades de un punto. Por otro lado, las coordenadas esféricas descomponen la posición en el radio tridimensional, el ángulo de coaltitud y el ángulo azimutal. En este contexto de nubes de puntos 3D almacenadas en *Octrees*, se tratará como el origen el centro geométrico de cada nodo.

Los reordenamientos en cuestión consisten en almacenar los puntos dentro de un nodo terminal de la estructura ordenados según alguna característica de las

mencionadas. Así, se podrán calcular los límites de forma teórica para ese valor que puede tener un punto de forma que satisfaga la restricción de vecindad, dada una búsqueda con un kernel determinado.

Se estudiará el impacto en materia de eficiencia temporal de la aplicación de esta poda teórica de puntos en la implementación del *Octree* lineal propuesta en [1] con respecto a los algoritmos usados inicialmente. Finalmente, también se analizará la escalabilidad de la técnica, paralelizando la búsqueda de vecinos.

Capítulo 2

Estado de conocimiento del problema a abordar

El uso de Octrees como estructura de datos en la gestión y almacenamiento de nubes de puntos 3D es actualmente predominante. Existen librerías ampliamente reconocidas que dan soporte nativo tanto a Octrees como a KDTrees a partir de nubes de puntos.

Una de estas librerías es Open3D. Está escrito en C++ pero tiene soporte para numerosos lenguajes. De hecho, se considera el estándar en Python para la visualización 3D. Incluye métodos de cálculos de distancia y búsqueda de vecinos, tanto mediante kernels de búsqueda como mediante la técnica de *K nearest neighbors* (KNN).

Por otra parte, la librería de referencia absoluta en el entorno académico e industrial para el procesamiento de geometría 3D en C++ es PCL (Point Cloud Library) [ref pcl]. PCL proporciona un ecosistema masivo y modular que da soporte nativo a estructuras espaciales mediante las clases `pcl::Octree` y `pcl::KdTree`. El enfoque de PCL en sus Octrees tradicionales se basa en una estructura dinámica de punteros. Esto permite modificaciones en tiempo real sobre la nube (como la inserción o eliminación dinámica de puntos y la detección de cambios espaciales), pero introduce una penalización en el rendimiento de la memoria a causa de la incoherencia entre la localidad espacial y la localidad en memoria, provocando constantes saltos de punteros en el direccionamiento de la caché de la CPU durante los recorridos y las búsquedas.

Un aspecto crítico que comparten tanto Open3D como PCL es que no implementan sus algoritmos de búsqueda de puntos cercanos desde cero, sino que integran internamente la librería FLANN (Fast Library for Approximate Nearest Neighbors) [ref flann]. Se trata de una biblioteca optimizada para la búsqueda de vecinos más cercanos en espacios vectoriales multidimensionales. Su principal característica es la capacidad de realizar búsquedas aproximadas (Approximate Nearest Neighbor o ANN), lo que permite sacrificar un porcentaje marginal de precisión matemática exacta a cambio de acelerar drásticamente el tiempo de

6CAPÍTULO 2. ESTADO DE CONOCIMIENTO DEL PROBLEMA A ABORDAR

computación.

Otra librería relevante con el mismo propósito, escrita también en C++, es la de PicoTree. A partir de nubes de puntos genera un KDTree en un array contiguo mediante la indexación de puntos. También permite recorridos de búsqueda de KNN y esféricos.

Por otra parte, las curvas de llenado también son ampliamente aplicadas a estructuras jerárquicas para tareas de indexado o compresión. La integración de códigos de Morton en estructuras de Octree lineales ha sido una estrategia clave para mejorar la eficiencia en la ordenación de puntos. Un ejemplo de ello es el trabajo de Behley et al. (implementación del `unibnOctree`) [2], donde se emplean para reducir la necesidad de recorrer el árbol en toda su profundidad durante las búsquedas de vecindad, logrando superar el rendimiento de otras librerías populares de la disciplina. Estos códigos son reconocidos por su simplicidad algorítmica y rapidez de cómputo. Sin embargo, el otro gran algoritmo de SFC, las curvas de Hilbert, han ganado popularidad por garantizar la preservación de la localidad. Es decir, las curvas de Hilbert aseguran que si dos puntos son cercanos en el espacio tridimensional, estarán próximos en la proyección unidimensional.

No obstante, la aplicación directa de SFC en la búsqueda de vecindades en nubes de puntos 3D obtenidas con sensores LiDAR fue implementada originalmente en el trabajo de Pablo Díaz et al.[1]. En primer lugar se propone una estructura Octree de punteros estándar. Se construye mediante fragmentaciones recursivas. Es cierto que se le pueden aplicar reordenamiento según curvas de Morton o Hilbert para mejorar la cercanía en memoria de los puntos próximos, pero este enfoque sigue presentando un gran problema en los accesos a memoria. Cada vez que se cambia de nodo para evaluar, se salta a un nuevo espacio de memoria que puede estar totalmente disperso, provocando continuamente fallos en la memoria caché.

Por este motivo se implementó la versión lineal del Octree, que almacena todos los puntos en un único vector contiguo, pensado de forma que todos los puntos de una hoja están consecutivos en memoria, además de buscar que áreas cercanas estén también cercanas en las posiciones del array. Adicionalmente, también se incluyó un nuevo algoritmo (para la búsqueda mediante kernel geométrico) con una funcionalidad adicional que acelera el proceso. El algoritmo clásico, usado también en el Octree de punteros, recorre los nodos desde la raíz, comprobando en cada paso si el nodo actual interseca con el kernel de búsqueda. Este método, sin embargo, diferencia el caso de que el kernel contenga completamente al área que cubre el nodo, ya sea terminal o no. Si es así, añade todos los puntos que se encuentran en él. En caso contrario, continúa con la búsqueda en profundidad.

Sobre este punto se elabora el presente trabajo. En la publicación de [1] así como en la amplia mayoría de librerías conocidas hay un comportamiento común en el diseño de las estructuras de indexación espacial: los modelos justifican su eficiencia en algoritmos de búsqueda en la aplicación sistemática de la poda (*pruning*) a nivel de nodos intermedios. Usan funciones de comprobación de solapa-

miento geométrico para descartar de forma temprana subárboles completos cuyo volumen geométrico no interseca con el kernel de vecindad. Sin embargo, una vez alcanzadas las hojas de la estructura, se evalúa la distancia euclídea de todos los puntos con el punto de consulta de manera individual.

Hasta donde se tiene constancia en la literatura técnica y en las soluciones de código abierto analizadas, no existe ninguna implementación que acote el rango de puntos a evaluar mediante una poda matemática interna a nivel de nodo hoja.

Este cuello de botella constituye la principal motivación y el núcleo de la contribución de este trabajo. Al introducir funciones de cálculos de rango basadas en reordenamientos geométricos locales (cartesianos y esféricos), este estudio permite aislar zonas y rangos de puntos dentro de los nodos hoja, anteriormente concebidos como unidades indivisibles, buscando un aumento en el rendimiento temporal de la estructura del Octree lineal. Adicionalmente, también se está aportando una mayor flexibilidad a la hora de diseñar el tamaño de los nodos.

8CAPÍTULO 2. ESTADO DE CONOCIMIENTO DEL PROBLEMA A ABORDAR

Capítulo 3

Materiales

En esta sección se definirán y precisarán todas las herramientas, métodos, algoritmos y equipos que fueron empleados y tomados para la realización del estudio.

Nube	nPuntos	Dataset	Descripción
<i>bildstein_station1_xyz_intensity_rgb</i>	29.6 M	Semantic3D	Escaneo LiDAR terrestre y urbano desde una posición estática, muy denso.
<i>sg27_station8_intensity_rgb</i>	73.9 M	Semantic3D	Escaneo LiDAR.
<i>5145_54340</i>	12.1 M	DALES	LiDAR aéreo obtenido desde aeronaves. Está dividido en celdas disjuntas.
<i>Lille_0</i>	10.0 M	Paris-Lille 3D	Escaneo .
<i>Paris-Luxembourg_6</i>	10.0 M	Paris-Lille 3D	Sensor LiDAR sobre un vehículo (MLS) que mapea dos calles en París.
<i>PNOA_2024_PNR_489-4672_NPC01</i>	18.5 M	PNOA-II	Cobertura del territorio español mediante LiDAR aéreo. Tiene baja densidad.
<i>heiss</i>		Hessigheim 3D	Dataset aéreo capturado mediante UAV (<i>Unmanned Aerial Vehicle</i>) del pueblo alemán de Hessigheim

Cuadro 3.1: Mapa de nubes utilizadas en las evaluaciones experimentales.

Capítulo 4

Metodología

Capítulo 5

Pruebas

Capítulo 6

Discusión de los resultados

Comprobación da(s) hipótese(s) de partida do traballo. Comparativa con algunha das solucións similares presentadas no estado de coñecemento, resaltando por que o traballo presentado mellora as solucións anteriores.

Capítulo 7

Exemplos (eliminar capítulo na versión final)

7.1. Un exemplo de sección

Esta é *letra cursiva*, esta é **letra negrilla**, esta é letra subrallada, e esta é **letra curier**. Letra tiny, scriptsize, small, large, Large, LARGE e moitas más. Exemplo de fórmula: $a = \int_0^\infty f(t)dt$. E agora unha ecuación aparte:

$$S = \sum_{i=0}^{N-1} a_i^2. \quad (7.1)$$

As ecuaciones se poden referenciar: ecuación (7.1).

7.1.1. Un exemplo de subsección

O texto vai aquí.

7.1.2. Otro exemplo de subsección

O texto vai aquí.

Un exemplo de subsubsección

O texto vai aquí.

Un exemplo de subsubsección

O texto vai aquí.

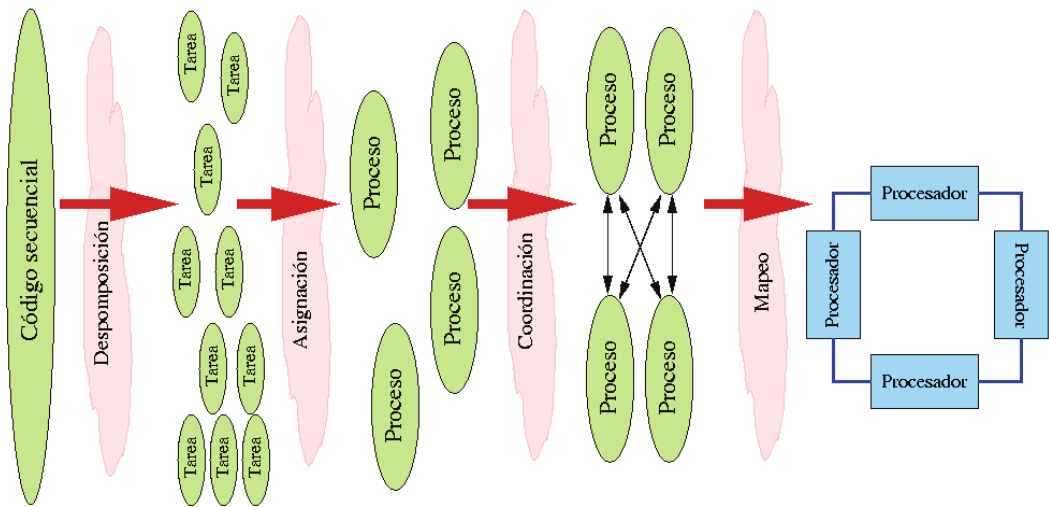


Figura 7.1: Esta é a figura de tal e cal.

Izquierda	Derecha	Centrado
ll	r	cccc
llll	rrr	c

Cuadro 7.1: Esta é a táboa de tal e cal.

Un exemplo de subsubsección

O texto vai aquí.

7.2. Exemplos de figuras e cadros

A figura número 7.1.
O cadro (taboa) número 7.1.

7.3. Exemplos de referencias á bibliografía

Este é un exemplo de referencia a un documento descargado da web [1]. E este é un exemplo de referencia a unha páxina da wikipedia [2]. Agora un libro [3] e agora unha referencia a un artigo dunha revista [4]. Tamén se poden por varias referencias á vez [1, 3].

7.4. Ejemplos de enumeraciones

Con puntos:

- Un.
- Dous.
- Tres.

Con números:

1. Catro.
2. Cinco.
3. Seis.

Exemplo de texto verbatim:

```
0 texto          verbatim
  se visualiza tal
    como se escribe
```

Exemplo de código C:

```
#include <math.h>
main()
{  int i, j, a[10];
   for(i=0;i<=10;i++) a[i]=i; // comentario 1
   if(a[1]==0) j=1; /* comentario 2 */
   else j=2;
}
```

Exemplo de código Java:

```
class HelloWorldApp {
    public static void main(String[] args) {
        System.out.println("Hello World!"); // Display the string.
    }
}
```


Capítulo 8

Conclusións e posibles ampliacións

Resumo das principais aportacións do traballo, explicacións das suposicións e limitacións do traballo, e posibles vías de mellora.

Apéndice A

Manuais técnicos

En función do tipo de Traballo e metodoloxía empregada, o contido poderase dividir en varios documentos. En todo caso, neles incluírase toda a información precisa para aquelas persoas que desexen repetir o experimento (por exemplo código fonte, recursos necesarios, operacións necesarias para modificacións e probas, posibles problemas, etc.). O código fonte poderase entregar en soporte informático en formatos PDF ou postscript.

Apéndice B

Manuais de usuario

Incluirán toda a información precisa para aquelas persoas que repliquen o experimento: instalación, utilización, configuración, mensaxes de erro, etc. A documentación do usuario debe ser autocontida, é dicir, para o seu entendemento o usuario final non debe precisar da lectura doutro manual técnico.

Apéndice C

Licenza

Se se quere pór unha licenza (GNU GPL, Creative Commons, etc), o texto da licenza vai aquí.

Bibliografía

- [1] Nvidia CUDA programming guide. Versión 2.0, 2010. Disponible en <http://www.nvidia.com>.
- [2] Acceso múltiple por división de código. Artigo da wikipedia (<http://es.wikipedia.org>). Consultado o 2 de xaneiro do 2010.
- [3] R.C. Gonzalez e R.E. Woods, *Digital image processing*, 3ª edición, Prentice Hall, New York, 2007.
- [4] P. González, J.C. Cartex e T.F. Pellas, “Parallel computation of wavelet transforms using the lifting scheme”, *Journal of Supercomputing*, vol. 18, no. 4, pp. 141-152, junio 2001.